

From Representation to Clusters: A Contrastive Learning Approach for Attributed Hypergraph Clustering

Li Ni

School of Computer Science and Technology
Anhui University
Hefei, China
nili@ahu.edu.cn

Lin Mu*

School of Computer Science and Technology
Anhui University
Hefei, China
mulin@ahu.edu.cn

Shuaikang Zeng

School of Computer Science and Technology
Anhui University
Hefei, China
e24301218@stu.ahu.edu.cn

Longlong Lin

School of Computer Science and Technology
Southwest University
Chongqing, China
longlonglin@swu.edu.cn

Abstract

Contrastive learning has demonstrated strong performance in attributed hypergraph clustering. Typically, existing methods based on contrastive learning first learn node embeddings and then apply clustering algorithms, such as k-means, to these embeddings to obtain the clustering results. However, these methods lack direct clustering supervision, risking the inclusion of clustering-irrelevant information in the learned graph. To this end, we propose a Contrastive learning approach for Attributed Hypergraph Clustering (CAHC), an end-to-end method that simultaneously learns node embeddings and obtains clustering results. CAHC consists of two main steps: representation learning and cluster assignment learning. The former employs a novel contrastive learning approach that incorporates both node-level and hyperedge-level objectives to generate node embeddings. The latter joint embedding and clustering optimization to refine these embeddings by clustering-oriented guidance and obtains clustering results simultaneously. Extensive experimental results demonstrate that CAHC outperforms baselines on eight datasets.

CCS Concepts

• Theory of computation → Unsupervised learning and clustering; • Mathematics of computing → Hypergraphs.

Keywords

Attributed hypergraph clustering, contrastive learning, hypergraph representation learning, hypergraph attention network

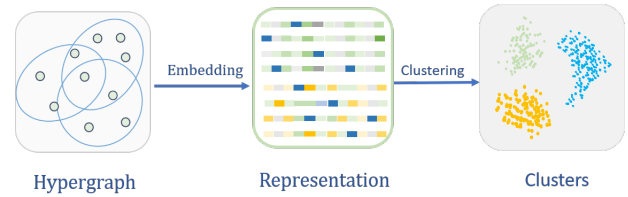
ACM Reference Format:

Li Ni, Shuaikang Zeng, Lin Mu, and Longlong Lin. 2026. From Representation to Clusters: A Contrastive Learning Approach for Attributed Hypergraph Clustering. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*.

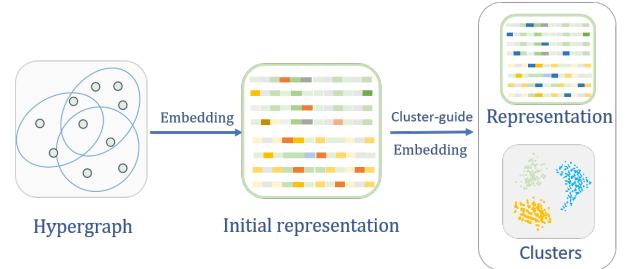
*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792432>



(a) Existing contrastive hypergraph clustering methods.



(b) Proposed CAHC.

Figure 1: Comparison of contrastive learning-based hypergraph clustering methods and the proposed CAHC.

April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792432>

1 Introduction

Hypergraphs are an effective tool for modeling high-order relationships among entities, as each hyperedge can connect several nodes [5, 49]. This ability makes hypergraphs particularly useful in modeling complex relationships in various real-world applications, such as recommender systems [39], computer vision [47], and neuroscience [41]. These high-order relationships cannot be effectively represented by ordinary pairwise graphs, where each edge connects only two nodes [4, 5, 49, 51]. Hypergraphs often contain cluster structure [18], and identifying such structures has widespread applications in fields [1], such as parallel computing [15] and circuit design [17]. As a result, researchers have developed various hypergraph clustering methods [22, 34, 40].

Existing hypergraph clustering methods can be broadly categorized into traditional and deep learning-based approaches [34]. Traditional methods include spectral clustering-based methods [29, 45], fuzzy memberships-based methods [40], and stochastic block models [7, 28, 48]. These methods often struggle to capture complex and high-order structural information effectively [22]. In contrast, deep learning-based approaches have gained much attention for their ability to capture such high-order information [34]. Several recent self-supervised frameworks, including SE-HSSL [22], HyperGCL [36], and TriCL [20] have improved representation capabilities. These methods generally use contrastive learning to learn representations, which are then partitioned using clustering algorithms, such as k-means, to form clusters, as shown in Figure 1(a).

Although contrastive learning-based hypergraph clustering methods have achieved promising results, these methods typically learn representations by comparing two augmented views, lacking explicit clustering guidance. As a result, they may incorporate irrelevant or non-cluster-related features into the learned graph, ultimately resulting in low-quality clusters. To address this, we propose a contrastive learning approach for attributed hypergraph clustering, called CAHC. As shown in Figure 1(b), CAHC comprises two steps: the representation learning and the cluster assignment learning. The former aims to learn high-quality node embeddings based on the hypergraph structure and attributes. The latter jointly optimizes both the embeddings and clustering, achieving a synergistic optimization of both the node embeddings and clustering results. Notably, in CAHC, traditional clustering algorithms such as k-means are no longer required. The main contributions of this work are summarized as follows:

- We propose an attributed hypergraph clustering method based on contrastive learning, termed CAHC. To the best of our knowledge, it is the first end-to-end model for attributed hypergraph clustering. CAHC employs contrastive learning for node embeddings, followed by a joint optimization of embedding and clustering objectives to yield the clustering results.
- We propose a clustering loss function that measures the closeness between soft assignments and hard assignments. It shares node embeddings with representation learning to effectively implement cluster-guidance embedding and cluster assignment simultaneously. In addition, we design a novel hyperedge-level objective to capture the structural information of hypergraphs.
- Extensive experiments on eight real-world datasets demonstrate that CAHC outperforms baselines. Furthermore, ablation studies confirm that each component of CAHC plays a crucial role.

Reproduction. The source code of this paper can be found at <https://github.com/nilics/CAHC>.

2 Proposed method

2.1 Problem statement

Data model. We denote a hypergraph $\mathcal{H} = (V, E, X)$, where $V = \{v_1, v_2, \dots, v_N\}$ represents the set of nodes, with N being the total number of nodes. The set $E = \{e_1, e_2, \dots, e_M\}$ represents the

collection of hyperedges, where each hyperedge $e_j \subseteq V$ is a non-empty subset of nodes and M is the number of hyperedges. Each node $v_i \in V$ is associated with a feature vector $\mathbf{x}_i \in \mathbb{R}^d$, which is stacked into the node feature matrix $X \in \mathbb{R}^{N \times d}$, where each row $X[i, :]$ corresponds to the feature vector of node v_i . The hyperedges are also represented by the incident matrix $H \in \{0, 1\}^{N \times M}$, where $H_{ij} = 1$ if node v_i is part of hyperedge e_j , and 0 otherwise. Based on the incident matrix H , the hypergraph $\mathcal{H} = (V, E, X)$ can also be expressed as $\mathcal{H} = (X, H)$.

Problem statement. Given a hypergraph $\mathcal{H} = (X, H)$, the task aims to simultaneously encode the nodes in an unsupervised manner using a hypergraph neural network, while also partitioning the hypergraph into k clusters $C = \{C_1, C_2, \dots, C_k\}$ without any supervision.

2.2 Overview

In this section, we propose a contrastive learning approach for attributed hypergraph clustering, called CAHC. The process of CAHC is shown in Figure 2. CAHC comprises two core steps: representation learning, which aims to learn node embeddings from the inherent structural and attribute information; and cluster assignment learning, which optimizes these embeddings to align with the final cluster structure, thereby achieving synergistic optimization of representation learning and clustering tasks.

2.3 Representation learning

CAHC adopts a contrastive learning paradigm that learns representations by maximizing the agreement between different views. As depicted in Figure 2, it comprises three main components: data augmentation, hypergraph encoding, and objective optimization. First, data augmentation is applied to the hypergraph $\mathcal{H} = (X, H)$ to obtain two correlated views $\mathcal{H}_1 = (X^{(1)}, H^{(1)})$ and $\mathcal{H}_2 = (X^{(2)}, H^{(2)})$. Next, a shared hypergraph neural network encoder f_θ generates node representations Z for these augmented views. Finally, the encoder is optimized using two learning objectives: a hyperedge-level contrastive objective and a node-level contrastive objective.

Data augmentation. We consider two hypergraph augmentation strategies: node feature masking and membership relation masking. In node feature masking, we construct a binary masking matrix $M_f \in \{0, 1\}^{N \times d}$ with the same shape as the node feature matrix X [31, 46]. Each element $M_{f,ij}$ is independently sampled from a Bernoulli distribution: $M_{f,ij} \sim \mathcal{B}(1 - p_f)$, where p_f represents the probability of masking a feature element [20]. The augmented feature matrix X' is computed as follows:

$$X' = X \odot M_f, \quad (1)$$

where $\odot$ denotes element-wise multiplication. This operation preserves feature values where $M_{f,ij} = 1$, and masks (sets to zero) features where $M_{f,ij} = 0$.

Membership relation masking is designed to perturb higher-order relationships by randomly removing or adding nodes in hyperedges. Similar to the node feature masking strategy, we construct a binary masking matrix $M_m \in \{0, 1\}^{N \times M}$ with the shape of the incident matrix H . Each element $M_{m,ij}$ is independently sampled from a Bernoulli distribution $\mathcal{B}(1 - p_m)$, where p_m denotes the probability of masking a node-hyperedge association [22]. The enhanced

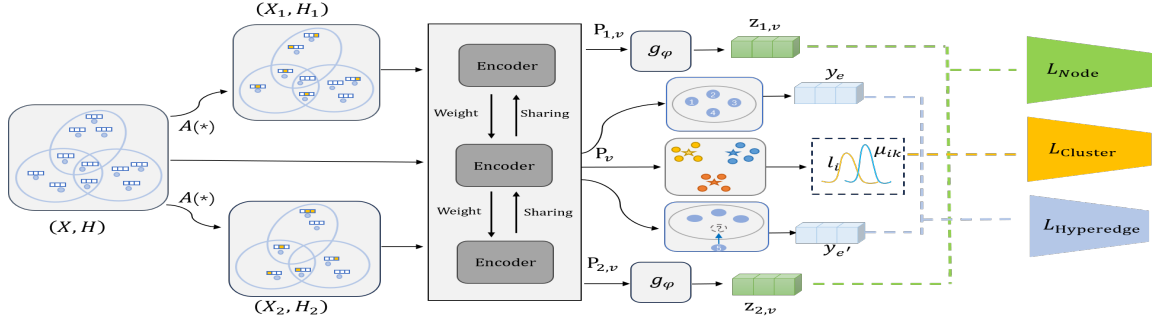


Figure 2: Overview of CAHC. CAHC consists of two core steps: **hypergraph representation learning** and **cluster assignment learning**. Specifically, hypergraph representation learning optimizes $\mathcal{L}_{\text{hyper}} + \mathcal{L}_{\text{node}}$ to learn node embeddings from the inherent structural and attribute information of the data. Cluster assignment learning optimizes $\mathcal{L}_{\text{hyper}} + \mathcal{L}_{\text{node}} + \mathcal{L}_{\text{clu}}$ to achieve representations and clustering results.

hypergraph topology is the modified incident matrix H' :

$$H' = H \odot M_m, \quad (2)$$

where $\odot$ denotes element-wise multiplication. This operation preserves the elements where $M_{m,ij} = 1$, while masking those where $M_{m,ij} = 0$. The hyperparameter p_m controls the intensity of this topological perturbation.

Hypergraph encoder. The hypergraph encoder is based on a hypergraph neural network (HGNN). To capture the varying importance of nodes within a hyperedge, we incorporate a multi-head attention mechanism into the HGNN. Formally, the l th layer of the hypergraph neural network can be expressed as:

$$P^{(l)} = \sigma \left(\text{Agg}_{E \rightarrow V}(P^{(l-1)}, Q^{(l-1)}, H) \Theta_V^{(l)} + P^{(l-1)} \Phi_V^{(l)} \right), \quad (3)$$

$$Q^{(l)} = \sigma \left(\text{Agg}_{V \rightarrow E}(P^{(l-1)}, Q^{(l-1)}, H) \Theta_E^{(l)} + Q^{(l-1)} \Phi_E^{(l)} \right), \quad (4)$$

where σ is the nonlinear activation function ReLU, Θ_V , Φ_V , Θ_E , and Φ_E are learnable linear transformation matrices. $P^{(l)}$ and $Q^{(l)}$ represent node and hyperedge representations, respectively. $P^{(0)}$ is initialized with node features X , and $Q^{(0)}$ is initialized by aggregating the node features X of its associated nodes (e.g., via mean pooling). The functions $\text{Agg}_{V \rightarrow E}$ and $\text{Agg}_{E \rightarrow V}$ are aggregation operations, where $\text{Agg}_{E \rightarrow V}$ refers to hyperedge-to-node aggregation and $\text{Agg}_{V \rightarrow E}$ corresponds to node-to-hyperedge aggregation.

Aggregation functions $\text{Agg}_{V \rightarrow E}$ and $\text{Agg}_{E \rightarrow V}$ are based on multi-head attention mechanisms to assign different weights to distinct node and hyperedge representations adaptively. This mechanism addresses a key limitation of HGNNs, which aggregate information via simple averaging and thus fail to model the varying importance of nodes within a hyperedge. Taking the hyperedge-to-node aggregation $\text{Agg}_{E \rightarrow V}$ as an example, it is computed as follows:

$$\text{Agg}_{E \rightarrow V}(P, Q, H) = \big\|_{h=1}^{N_h} \left(\alpha_{EV}^{(h)} Q W_Q^{(h)} \right) W_O, \quad (5)$$

where $\big\|$ denotes the concatenation operation, N_h is the number of attention heads, $W_Q^{(h)}$, and W_O are learnable weight matrices. The attention coefficient matrix $\alpha_{EV}^{(h)}$ is computed as follows to learn

the attention of node v_i to its associated hyperedge e_j :

$$\alpha_{EV,ij}^{(h)} = \text{softmax}_j(s_{ij}^{(h)}) = \frac{\exp(s_{ij}^{(h)})}{\sum_{j' \in \mathcal{N}_V(i)} \exp(s_{ij'}^{(h)})}, \quad (6)$$

$$s_{ij}^{(h)} = \text{LeakyReLU} \left((a^{(h)})^\top [p_i W_P^{(h)} \| q_j W_Q^{(h)}] \right), \quad (7)$$

where $\mathcal{N}_V(i)$ denotes the set of hyperedges associated with node v_i . The term $s_{ij}^{(h)}$ represents the unnormalized attention score of node v_i towards hyperedge e_j under the h -th attention head, which is parameterized by a learnable attention vector $a^{(h)}$. The aggregation from nodes to hyperedges, denoted as $\text{Agg}_{V \rightarrow E}$, is implemented symmetrically.

Contrastive loss. Based on the aforementioned data augmentation strategy and shared hypergraph encoder, we optimize the encoder by minimizing the following two loss functions: hyperedge-level loss $\mathcal{L}_{\text{hyper}}$ and node-level loss $\mathcal{L}_{\text{node}}$.

The hyperedge-level loss aims to enable the model to distinguish which hyperedges are real or generated negative hyperedges, as follows:

$$\mathcal{L}_{\text{hyper}} = -\frac{1}{|E|} \sum_{e \in E} \log(\sigma(S(y_e))) - \frac{1}{|E_{\text{neg}}|} \sum_{e \in E_{\text{neg}}} \log(1 - \sigma(S(y_e))), \quad (8)$$

where $\sigma(\cdot)$ denotes the Sigmoid function, $S(\cdot)$ represents a shared scoring function (implemented as a multilayer perceptron) that computes a scalar score for all hyperedge representations, y_e is the representation of hyperedge e , obtained by applying mean pooling to the node embeddings (generated by the encoder) that belong to e , E represent hyperedges in the original hypergraph, and E_{neg} represents the set of generated negative hyperedges. These negative hyperedges are constructed by randomly replacing nodes in each original hyperedge $e \in E$ to form invalid ones. The hyperedge-level loss adheres to the principle of contrastive learning by comprising two opposing objectives: a positive term that maximizes the similarity for nodes connected by real hyperedges in E , and a negative term that minimizes the similarity for nodes connected by artificially constructed negative hyperedges in E_{neg} .

Node-level loss aims to make the representation of the same node in two augmented views similar, while ensuring that these

representations are distinguishable from those of other nodes. For augmented views $\mathcal{H}_1 = (X^{(1)}, H^{(1)})$ and $\mathcal{H}_2 = (X^{(2)}, H^{(2)})$, the symmetric node loss function $\mathcal{L}_{\text{node}}$ is computed as follows:

$$\mathcal{L}_{\text{node}} = \frac{1}{2N} \sum_{i=1}^N \left[\ell(z_i^{(1)}, z_i^{(2)}) + \ell(z_i^{(2)}, z_i^{(1)}) \right], \quad (9)$$

where $\ell(z_i^{(1)}, z_i^{(2)})$ is computed as:

$$\ell(z_i^{(1)}, z_i^{(2)}) = -\log \frac{\exp(\text{sim}(z_i^{(1)}, z_i^{(2)})/\tau_n)}{\sum_{k=1}^N \exp(\text{sim}(z_i^{(1)}, z_k^{(2)})/\tau_n)}, \quad (10)$$

where $\text{sim}(\cdot, \cdot)$ denotes cosine similarity, and τ_n is a temperature hyperparameter, $z_i^{(1)}$ and $z_i^{(2)}$ represent the final node representation vectors obtained for the same node v_i in two augmented views, respectively, which through a shared encoder f_θ and a nonlinear projection head g_ϕ . Here, for any node representation $z_i^{(1)}$ from the first view, its corresponding representation $z_i^{(2)}$ in the second view forms a positive sample pair. Meanwhile, $z_i^{(1)}$ forms negative sample pairs with all other node representations $z_j^{(2)}$ ($j \neq i$) from the second view[20].

The hyperedge-level loss calculated by Eq. (8) and node-level loss calculated by Eq. (9) are complementary. The hyperedge loss forces the model to understand the complex, higher-order interaction patterns between nodes, focusing on the structural within the hypergraph. On the other hand, the node loss ensures that the representations of different nodes are sufficiently distinct, emphasizing the individual discriminability of nodes. Together, these losses are integrated into the final contrastive loss, calculated as:

$$\mathcal{L}_{\text{rep}} = \mathcal{L}_{\text{hyper}} + \mathcal{L}_{\text{node}}. \quad (11)$$

CAHC jointly optimizes Eq. (11) during the representation learning phase, enabling the learned node embeddings to simultaneously preserve information at both the hyperedge and node levels.

2.4 Cluster assignment learning

Embedding methods without clustering supervision may learn non-cluster-related features, resulting in poor clustering results. To provide clustering-oriented guidance, CAHC performs joint optimization of the embedding and clustering, thereby obtaining the representation and the final cluster assignments simultaneously.

CAHC assumes that there are K clusters, denoted by $c = \{c_1, c_2, \dots, c_K\}$. It computes the membership degree μ_{ik} for each node i with respect to each cluster center c_k , forming a soft assignment matrix. It also assigns each node to its nearest cluster center, generating high-confidence pseudo-labels l_i (i.e., hard assignments). The loss function is designed to minimize the discrepancy between the soft assignment and hard assignments, as follows:

$$\mathcal{L}_{\text{clus}} = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K \mathbb{I}(l_i = k) \log \mu_{ik}, \quad (12)$$

where $\mathbb{I}(\cdot)$ denotes the indicator function and soft assignment of node i to cluster c_k is computed as follows:

$$\mu_{ik} = \frac{\exp(\text{sim}(z_i, c_k)/\tau_c)}{\sum_{k'=1}^K \exp(\text{sim}(z_i, c_{k'})/\tau_c)}, \quad (13)$$

Algorithm 1 CAHC

Require: Node features X , hypergraph incidence matrix H , number of clusters K , representation learning epochs T_1 , cluster assignment learning epochs T_2 , loss weights α, β, γ .

Ensure: Final node cluster assignments Y .

```

1: Stage 1: Representation Learning
2: for  $epoch = 1$  to  $T_1$  do
3:   Generate two augmented views  $\mathcal{H}_1$  and  $\mathcal{H}_2$ .
4:   Construct negative hyperedges  $E_{\text{neg}}$ 
5:   Compute loss  $\mathcal{L}_{\text{rep}}$  with Eq. (11).
6:   Update the encoder  $f_\theta$  and projection head  $g_\phi$ 
7: end for
8: Obtain node representation  $Z_{\text{init}}$ 
9: Stage 2: Cluster assignment learning
10: Cluster centroids  $c \leftarrow k\text{-means}(Z_{\text{init}})$ 
11: for  $epoch = 1$  to  $T_2$  do
12:   Compute loss  $\mathcal{L}_{\text{rep}}$  with Eq. (11).
13:   Compute the soft assignments  $Q$  with Eq. (13)
14:   Compute pseudo-labels  $\hat{Y}$  with Eq. (14)
15:   Compute clustering loss  $\mathcal{L}_{\text{clus}}$  with Eq. (12)
16:   Update all model parameters by  $\mathcal{L}$  with Eq. (15)
17: end for
18: Obtain clustering results with  $Y$  from Eq. (14)
19: return  $Y$ 

```

where τ_c is a temperature hyperparameter, z_i represents node embedding of node i , and cluster centers $c = \{c_1, c_2, \dots, c_K\}$ are obtained by k-means [13]. The pseudo-label l_i is computed as:

$$l_i = \arg \max_k \mu_{ik}. \quad (14)$$

The overall loss function $\mathcal{L}$ combines the clustering loss $\mathcal{L}_{\text{clus}}$ in Eq. (12) with the representation learning loss $\mathcal{L}_{\text{rep}}$ in Eq. (11):

$$\mathcal{L} = \mathcal{L}_{\text{clus}} + \mathcal{L}_{\text{rep}}. \quad (15)$$

Algorithm 1 summarizes the execution steps of CAHC. CAHC generates two augmented views to compute the node loss $\mathcal{L}_{\text{node}}$ according to Eq. (9). It constructs negative hyperedges E_{neg} to compute the hypergraph loss $\mathcal{L}_{\text{hyper}}$ in Eq. (8) and then computes the contrastive loss. During the learning phase, CAHC optimizes Eq. (11) to update the encoder f_θ and the projection head g_ϕ , learning representations. Based on the representations Z_{init} , the algorithm applies k-means to obtain the cluster centroids. Using Z_{init} and the cluster centroids, CAHC proceeds to optimize Eq. (15), performing joint embedding and clustering optimization to obtain both the learned representations and the clustering results. The clustering results are derived using the pseudo-labels l_i , which are computed according to Eq. (14).

3 Experiment

In this section, we first introduce the experimental settings, including the datasets, baselines, and evaluation metrics. Subsequently, we compare CAHC with the baselines, analyze the impact of clustering guidance, conduct ablation studies, and examine parameter sensitivity.

Table 1: Statistics of the datasets.

Dataset	# Nodes	# Hyperedges	# Features	# Classes
Cora-C	1,434	1,579	1,433	7
Citeseer	1,458	1,079	3,703	6
Pubmed	3,840	7,963	500	3
Cora-A	2,388	1,072	1,433	7
DBLP	41,302	22,363	1425	6
20NewsW100	16,242	100	100	4
Mushroom	8,124	298	22	2
NTU2012	2,012	2,012	100	67

3.1 Experiment settings

Datasets. We evaluate the performance of the algorithms on eight public datasets: Cora-C [30], Citeseer [30], Pubmed [30], Cora-A [26], DBLP [26], NTU2012 [6], 20NewsW100 [8], and Mushroom [8]. Statistics information about these datasets is shown in Table 1. For details regarding the descriptions of each dataset, refer to Appendix A.

Baselines. We compare CAHC with six baselines: three classical embedding methods (Node2vec [12], DGI [33] and Hyper2vec [14]), a graph learning method (RAGC [50]), and two hypergraph self-supervised learning methods (TriCL [20] and SE-HSSL [22]). Referring to [20, 21], for self-supervised methods designed for graphs (DGI and RAGC), we first convert hypergraphs to graphs via clique expansion and apply these methods to the converted graphs. For embedding-based baseline methods, the learned embeddings are fed into the k-means algorithm [13] for clustering. Refer to Appendix B for the detailed settings of CAHC and baselines.

Metrics. We employ four metrics to assess clustering performance: Accuracy (ACC) [24], Normalized Mutual Information (NMI) [52], Adjusted Rand Index (ARI) [35], and Macro-F1 Score [35]. For ACC and F1 scores, we utilize the Hungarian algorithm for label alignment [19, 42]. For all metrics, higher values indicate better clustering performance.

3.2 Experiment result

Table 2 demonstrates the clustering performance of all algorithms on eight public datasets. Overall, CAHC outperforms baselines on most datasets. It is observed that methods designed for hypergraphs (e.g., TriCL, SE-HSSL, and CAHC) outperform those designed for graphs (Node2Vec, DGI). Since the latter requires first converting hypergraphs to ordinary graphs via clique expansion, this conversion leads to information loss. Notably, CAHC outperforms TriCL and SE-HSSL on the Citeseer, Pubmed, and Mushroom datasets. Specifically, on the Pubmed dataset, CAHC achieves relative improvements of 10.3% and 17.1% in NMI and ARI, respectively, compared to TriCL and SE-HSSL. This is primarily because TriCL and SE-HSSL rely on a two-step process: first, learning embeddings, followed by clustering, which lacks effective clustering guidance. In contrast, our algorithm optimizes both representation learning and clustering in a unified, end-to-end manner.

RAGC gets the best performance on the Mushroom dataset, which contains only two clusters. Due to the small number of clusters, RAGC’s contrastive sample adaptive-differential awareness module generates highly accurate pseudo-labels. This reliability

allows the module to effectively discriminate and weight difficult samples, leading to well-optimized cluster boundaries. In contrast, on datasets with more clusters, such as Cora-C and Citeseer, the initial pseudo-labels are less accurate. On the 20NewsW100 dataset, the performance of CAHC is inferior to that of SE-HSSL. The negative hyperedge generation strategy employed by CAHC is not well-suited for hyperedges of large size, where each hyperedge connects, on average, more than 160 nodes in 20NewsW100. Specifically, the strategy of replacing only a single node to generate negative hyperedges fails to produce effective negative samples. This is because the generated ‘negative samples’ are nearly identical to the original samples, thus diminishing the discriminative power of the hyperedge-level loss.

3.3 Impact of clustering guidance

To investigate the effect of the clustering guidance, we design two variants of CAHC: (1) “CAHC + k-means”. This variant generates embeddings from CAHC and applies k-means to them to obtain the final results; and (2) “w/o Clus”. This variant first obtains embeddings from the representation learning step and then applies k-means to these embeddings to obtain the results. We evaluate CAHC + k-means, w/o Clus, and the original CAHC on the Pubmed, Cora-A, and DBLP datasets.

Figure 3 presents ACC, F1, NMI, and ARI of CAHC + k-means, w/o Clus, and CAHC. Figure 3 shows that CAHC + k-means outperforms w/o Clus, demonstrating that the clustering loss helps improve the embedding quality for the clustering task. Furthermore, Figure 3 indicates that ‘CAHC + k-means’ and CAHC are comparable. This is because the membership score μ_{ik} , calculated by Eq. (13), follows a similar principle as k-means, where each node is assigned to the nearest cluster center.

3.4 Ablation study

To validate components of CAHC, we conducted an ablation study on eight datasets. Specifically, five variants of CAHC are constructed: 1) “w/o re”. This variant represents CAHC without the representation learning step; 2) “w/o hy”. This variant represents CAHC without the hyperedge-level loss in Eq. (8); 3) “w/o no”. This variant represents CAHC without the node-level loss in Eq. (9); 4) “w/o cl”. This variant represents CAHC without the clustering module, where clustering is performed using k-means; and 5) “w/o mu”. This variant represents CAHC without the multi-head attention mechanism, using a standard HGNN [9] instead. Table 3 demonstrates the results. Overall, CAHC outperforms w/o re, “w/o hy”, “w/o no”, “w/o cl”, and “w/o mu”, indicating that the representation learning step, hyperedge-level loss, node-level loss, clustering module, and the encoder all play a crucial role. Afterward, we gradually analyze the impact of each component.

Effectiveness of contrastive objectives. The “w/o hy”, which removes hyperedge-level loss, performs significantly worse than CAHC on most datasets. For instance, on the Pubmed dataset, CAHC achieves an NMI score of 34.4, while the score of “w/o hy” is only 29.8. This empirically validates that explicitly learning from high-order structural information is a crucial component of the model’s success. Similarly, without the node-level contrastive loss (“w/o no”) also performs poorly. This is particularly evident on

Table 2: Comparison of clustering results (%) on datasets. All results are reported as mean $\pm$ std over 5 independent runs. The best results are in bold. ‘OOM’ denotes the out-of-memory failure, while ‘OOT’ signifies no results are obtained within 24 hours.

Dataset	Metric	Node2vec	DGI	RAGC	Hyper2vec	TriCL	SE-HSSL	CAHC
Cora-C	ACC	47.4 $\pm$ 5.4	68.4 $\pm$ 3.4	68.6 $\pm$ 1.6	59.7 $\pm$ 5.1	70.0 $\pm$ 1.1	72.0$\pm$3.5	71.4 $\pm$ 1.7
	F1	44.7 $\pm$ 7.1	62.0 $\pm$ 3.0	64.8 $\pm$ 1.6	58.5 $\pm$ 0.4	65.1 $\pm$ 1.8	67.0 $\pm$ 4.3	69.1$\pm$2.8
	NMI	32.5 $\pm$ 3.9	54.0 $\pm$ 1.0	50.9 $\pm$ 1.7	40.6 $\pm$ 1.1	55.2 $\pm$ 0.8	56.5$\pm$2.4	56.4 $\pm$ 0.9
	ARI	20.2 $\pm$ 4.2	47.1 $\pm$ 2.0	43.5 $\pm$ 2.2	33.6 $\pm$ 0.9	49.2 $\pm$ 0.2	49.3 $\pm$ 3.5	50.8$\pm$1.6
Citeseer	ACC	36.2 $\pm$ 1.6	63.8 $\pm$ 4.3	69.4 $\pm$ 1.4	42.7 $\pm$ 1.3	67.6 $\pm$ 1.5	68.2 $\pm$ 1.7	69.8$\pm$0.8
	F1	29.2 $\pm$ 3.1	55.7 $\pm$ 3.4	61.9 $\pm$ 1.9	41.1 $\pm$ 1.2	62.1 $\pm$ 2.0	62.5 $\pm$ 1.5	65.1$\pm$0.6
	NMI	20.0 $\pm$ 1.5	40.0 $\pm$ 2.1	43.5 $\pm$ 0.6	21.0 $\pm$ 1.0	44.1 $\pm$ 0.4	43.8 $\pm$ 2.1	45.9$\pm$1.2
	ARI	5.7 $\pm$ 0.6	38.6 $\pm$ 2.8	44.3 $\pm$ 1.7	16.9 $\pm$ 1.1	45.1 $\pm$ 0.3	43.3 $\pm$ 2.0	46.2$\pm$1.2
Pubmed	ACC	53.8 $\pm$ 0.5	63.9 $\pm$ 8.0	67.0 $\pm$ 2.5	69.4 $\pm$ 5.4	62.4 $\pm$ 0.6	65.8 $\pm$ 2.3	69.4$\pm$0.2
	F1	54.6 $\pm$ 0.5	61.8 $\pm$ 10.6	65.4 $\pm$ 2.2	67.7 $\pm$ 5.0	61.3 $\pm$ 0.6	65.1 $\pm$ 2.4	69.0$\pm$0.2
	NMI	21.6 $\pm$ 0.4	27.0 $\pm$ 6.4	27.1 $\pm$ 2.4	29.6 $\pm$ 3.3	29.7 $\pm$ 0.6	31.2 $\pm$ 1.1	34.4$\pm$0.4
	ARI	12.1 $\pm$ 0.5	24.9 $\pm$ 8.0	26.6 $\pm$ 3.1	33.5$\pm$6.1	24.7 $\pm$ 0.8	27.5 $\pm$ 2.2	32.2 $\pm$ 0.3
Cora-A	ACC	34.0 $\pm$ 2.0	60.9 $\pm$ 5.0	59.7 $\pm$ 1.1	22.7 $\pm$ 0.2	70.2 $\pm$ 1.5	61.0 $\pm$ 3.9	72.6$\pm$0.4
	F1	24.4 $\pm$ 2.2	53.5 $\pm$ 6.4	54.9 $\pm$ 1.8	17.8 $\pm$ 0.1	69.1 $\pm$ 1.2	52.3 $\pm$ 2.5	70.6$\pm$0.4
	NMI	12.5 $\pm$ 1.2	44.1 $\pm$ 1.6	42.2 $\pm$ 0.3	3.0 $\pm$ 0.5	50.2 $\pm$ 1.2	47.6 $\pm$ 1.9	53.1$\pm$0.7
	ARI	3.1 $\pm$ 1.2	37.0 $\pm$ 3.6	34.5 $\pm$ 0.9	0.6 $\pm$ 0.2	45.8 $\pm$ 2.5	38.8 $\pm$ 3.6	50.1$\pm$0.7
DBLP	ACC	39.91 $\pm$ 3.3				65.9 $\pm$ 4.1		72.3$\pm$8.0
	F1	38.05 $\pm$ 3.8	OOM	OOM	OOM	63.7 $\pm$ 5.3	OOT	70.4$\pm$8.6
	NMI	24.20 $\pm$ 1.9				62.4 $\pm$ 3.7		63.9$\pm$2.8
	ARI	9.63 $\pm$ 1.3				50.4 $\pm$ 5.2		58.2$\pm$5.9
20NewsW100	ACC	44.6 $\pm$ 1.2				61.5 $\pm$ 3.7	70.2$\pm$0.7	64.6 $\pm$ 3.1
	F1	41.8 $\pm$ 0.4	OOM	OOM	OOM	54.9 $\pm$ 2.7	65.0$\pm$1.0	63.2 $\pm$ 2.8
	NMI	11.1 $\pm$ 0.7				36.0 $\pm$ 3.8	39.1 $\pm$ 1.4	39.5$\pm$1.1
	ARI	12.1 $\pm$ 1.0				29.4 $\pm$ 9.4	40.4$\pm$0.2	35.6 $\pm$ 2.5
Mushroom	ACC	69.9 $\pm$ 6.5	70.7 $\pm$ 2.5	87.5$\pm$0.8		51.3 $\pm$ 1.4	78.3 $\pm$ 6.7	80.5 $\pm$ 2.4
	F1	69.3 $\pm$ 6.8	67.8 $\pm$ 3.1	87.4$\pm$0.8	OOM	38.8 $\pm$ 2.4	77.4 $\pm$ 7.6	80.5 $\pm$ 2.5
	NMI	15.0 $\pm$ 6.8	20.9 $\pm$ 4.8	46.6$\pm$2.8		3.4 $\pm$ 0.6	28.7 $\pm$ 10.5	30.3 $\pm$ 4.2
	ARI	17.6 $\pm$ 8.2	17.3 $\pm$ 3.8	56.3$\pm$2.5		0.5 $\pm$ 1.0	34.2 $\pm$ 14.4	37.5 $\pm$ 5.9
NTU2012	ACC	56.7 $\pm$ 2.2	65.0 $\pm$ 2.1	66.1 $\pm$ 0.3	58.8 $\pm$ 0.6	70.5 $\pm$ 0.5	70.5 $\pm$ 0.6	70.8$\pm$1.1
	F1	51.4 $\pm$ 1.7	59.8 $\pm$ 2.0	57.4 $\pm$ 1.1	54.0 $\pm$ 0.6	63.8 $\pm$ 0.8	64.0$\pm$0.7	63.7 $\pm$ 1.1
	NMI	77.3 $\pm$ 0.4	80.0 $\pm$ 0.6	79.9 $\pm$ 0.6	78.1 $\pm$ 0.1	83.1 $\pm$ 0.2	83.2 $\pm$ 0.3	83.8$\pm$0.2
	ARI	50.1 $\pm$ 2.5	59.4 $\pm$ 3.2	63.1 $\pm$ 1.2	51.8 $\pm$ 0.6	66.8 $\pm$ 0.5	66.9 $\pm$ 1.5	67.0$\pm$2.0

Cora-A, where the NMI score of CAHC is 53.1, while that of "w/o no" is only 45.4. This indicates that the individual discriminability of nodes is also indispensable for node representation.

Effectiveness of the end-to-end learning paradigm. "w/o cl" without clustering guidance performs worse than CAHC, indicating that such guidance is crucial for improving representations. "w/o cl" trains the encoder on the hypergraph without clustering guidance, leading to the incorporation of non-cluster-related features in the learned representations. Most NMI/ARI of "w/o re" are 0 because

"w/o re" assigns all nodes in the dataset to a single cluster, even when the number of clusters is set to be greater than one. It indicates that the representation learning step plays a crucial role.

Effectiveness of the proposed encoder architecture. "w/o mu" with a standard HGNN performs significantly worse than CAHC. Specifically, on the Cora-A dataset, the ARI score of "w/o mu" is 41.0, while the CAHC model achieves a score of 50.1, showing a considerable performance gap. Similarly, on the Citeseer dataset, the ARI of "w/o mu" is 42.0, and the ARI of CAHC is 45.9, indicating a

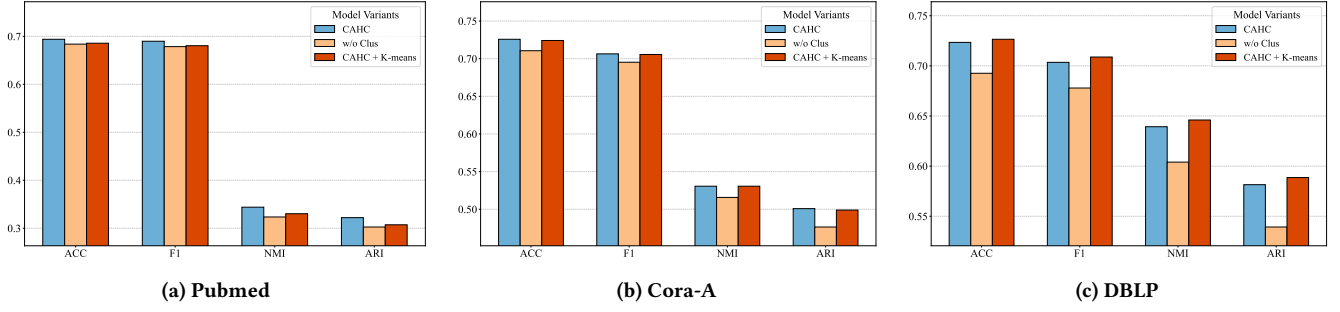


Figure 3: Impact of clustering guidance.

Table 3: Ablation study results (%).

Dataset	Metric	w/o re	w/o hy	w/o no	w/o cl	w/ mu	CAHC
Cora-C	ACC	28.0	68.8	72.9	70.4	71.6	71.4
	F1	6.2	63.4	70.5	68.4	67.1	69.1
	NMI	0.0	51.4	54.4	55.2	54.5	56.4
	ARI	0.0	46.5	51.9	49.2	49.9	50.8
Citeseer	ACC	21.7	69.1	65.5	68.8	67.0	69.8
	F1	5.9	64.9	60.6	64.3	62.4	65.1
	NMI	0.0	45.0	42.9	44.5	42.0	45.9
	ARI	0.0	44.7	42.1	44.7	42.5	46.2
Pubmed	ACC	40.7	61.6	69.2	68.4	70.4	69.4
	F1	19.3	60.4	68.4	67.9	69.7	69.0
	NMI	0.0	29.8	33.0	32.4	34.0	34.4
	ARI	0.0	25.2	30.5	30.3	33.1	32.2
Cora-A	ACC	30.2	69.8	65.1	71.1	65.9	72.6
	F1	6.7	68.4	59.3	69.5	63.7	70.6
	NMI	0.0	51.1	45.4	51.6	46.2	53.1
	ARI	0.0	46.6	42.3	47.6	41.0	50.1
DBLP	ACC	27.2	52.4	68.8	69.3	72.6	72.3
	F1	7.1	43.2	64.9	67.8	70.8	70.4
	NMI	0.0	52.3	60.9	60.4	63.0	63.9
	ARI	0.0	28.6	52.6	53.9	54.9	58.2
20newsW100	ACC	33.6	62.9	58.6	64.1	62.5	64.6
	F1	12.6	61.8	56.7	63.0	62.0	63.2
	NMI	0.0	39.6	33.8	39.2	36.9	39.5
	ARI	0.0	35.0	29.2	34.5	29.7	35.6
Mushroom	ACC	51.8	72.7	80.4	80.4	74.3	80.5
	F1	34.1	70.9	80.4	80.3	72.2	80.5
	NMI	0.0	21.2	30.2	29.8	26.5	30.3
	ARI	0.0	23.2	37.2	37.2	25.1	37.5
NTU2012	ACC	5.1	71.0	69.9	70.6	69.6	70.8
	F1	0.3	64.8	63.1	63.7	62.6	63.7
	NMI	0.3	83.8	83.4	83.7	82.4	83.8
	ARI	0.0	66.8	65.5	66.7	66.2	67.0

clear difference in performance. The results confirm that compared with HGNN, the multi-head attention mechanism-based HGNN could help CAHC to learn better representations.

3.5 Parameter sensitivity analysis

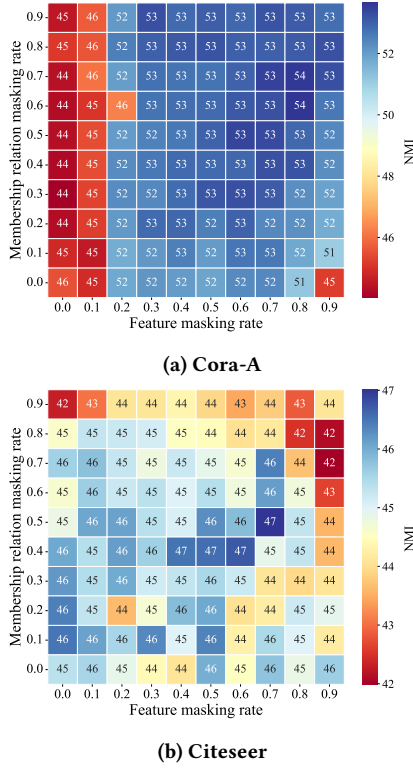
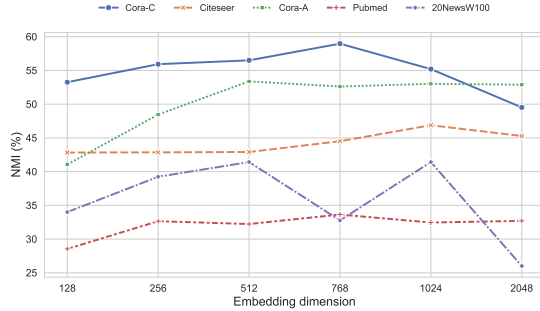
Here, we investigate the sensitivity of hyperparameters: feature masking rate p_f , the membership relation masking rate p_m , and embedding dimension D .

Effect of augmentation rates. The rates p_f and p_m ranged from 0.0 to 0.9 with a step size of 0.1. Figure 4 shows NMI of CAHC. A consistent conclusion can be drawn from both Cora-A and Citeseer datasets: CAHC achieves optimal performance at moderate values of p_f and p_m . Specifically, the model achieves its highest performance when the masking rates are within an intermediate range, roughly between 0.2 and 0.7 for both parameters. If the masking rates are too small (approaching 0.0), two overly similar views are generated. This provides an insufficient learning signal for the contrastive objective to develop a robust and discriminative encoder. Conversely, if the rates are too large (approaching 0.9), the underlying feature and structural information of the original hypergraph is broken, leading to a significant drop in performance. Therefore, intermediate augmentation rates are selected in experiments.

Effect of embedding dimension. The embedding dimension D is chosen from the set {128, 256, 512, 768, 1024, 2048}. Figure 5 illustrates NMI of CAHC. As D increases from 128 to an intermediate value (typically 512 or 768), the performance of CAHC improves overall. This suggests that CAHC benefits from a larger representation capacity to effectively capture the complex structural and feature information. However, we notice a subsequent decline in performance as the dimension further increases on most datasets. Taking 20NewsW100 as an example, which has only 100 original features, the model reaches its peak performance at an embedding dimension D of 512. The performance of CAHC deteriorates significantly when the dimension is increased to 2048. This indicates that datasets with limited features do not necessitate high-dimensional embeddings and may, in fact, suffer from excessively large dimensions. This phenomenon can be attributed to the embedding space becoming overly sparse, with many dimensions potentially becoming irrelevant or redundant. Thus, a medium-sized embedding dimension is suitable for most datasets. For datasets with high-dimensional features, a large dimension should be used.

4 Related work

Hypergraph representation learning. Hypergraph representation learning aims to embed the nodes (or hyperedges) of a hypergraph into a low-dimensional vector space. [2, 3, 23]. Due to the availability of labels, many studies have focused on semi-supervised learning for hypergraph representation [37, 43]. For example, Feng et al. [9] generalize graph convolutions to the hypergraph domain,

Figure 4: Sensitivity analysis of p_f and p_m .Figure 5: Sensitivity analysis of D .

with the core idea of using the hypergraph Laplacian operator for information propagation. Wu et al. [37] leverages spectral graph theory to capture the global latent structure within hypergraph learning. Yan et al. [44] propose an expansion method to transform a hypergraph into a standard graph and design a learning model to embed both hypervertices and hyperedges into a joint representation space. Qu et al. [25] propose a transformer-based framework that adopts a one-stage message passing paradigm.

These methods rely on labeled data as supervised signals, which may lead to high labeling costs and poor generalization. Recently, contrastive learning (CL) has gained attention for unsupervised hypergraph representation learning, as it does not require labels. For example, HyperGCL [36] addresses the critical issue of view

construction in hypergraph contrastive learning by designing both knowledge-based manual augmentations and generative augmentation methods. Li et al. [38] proposed the Hypergraph Contrastive Collaborative Filtering (HCCF) framework, which integrates hypergraph structure learning with cross-view contrastive learning. To further capture multi-granularity structural information, TriCL [20] employs a three-level contrastive framework involving nodes, hyperedges, and their membership relations. These studies primarily focus on learning node representations, requiring clustering algorithms like k-means for clustering tasks. Different from them, our work proposes an end-to-end approach that simultaneously learns node representations and generates clustering results.

Attributed hypergraph clustering. Attributed hypergraph clustering aims to use structure and attributes to discover communities or clusters in hypergraphs [11, 16, 34]. Existing methods can be broadly categorized into traditional and deep learning-based methods [34]. Traditional methods include modularity-based [10], fuzzy memberships-based method [40], and stochastic block model [7, 28, 48]. With the ability to capture high-order information, deep graph clustering, especially contrastive learning, has received increasing attention [11, 34]. For example, SE-HSSL [22], HyperGCL [36], and TriCL [20] use contrastive learning to learn representations, which are then partitioned using clustering algorithms, such as k-means, to form clusters. Additionally, Feng et al. [11] regard a multi-hop modularity as the objective function to obtain representation and use Louvain for clustering. Wang et al. [34] propose a hypergraph clustering network that utilizes hypergraph smoothing preprocessing instead of hypergraph convolution to avoid high computational complexity. Subsequently, k-means is employed to partition low-dimensional representations into several clusters. The above studies require clustering algorithms like k-means or Louvain to detect clusters. Unlike these studies, our work proposes an end-to-end clustering method that simultaneously obtains node representations and clustering results.

5 Conclusion

In this paper, we propose an unsupervised hypergraph clustering algorithm, CAHC. It consists of two main steps: hypergraph representation learning and cluster assignment learning. Hypergraph representation learning leveraging both node-level and hyperedge-level contrastive learning to obtain node embeddings. In addition, to capture the structural information of hypergraphs, we design a novel hyperedge-level objective that maximizes the similarity for nodes connected by real hyperedges and minimizes the similarity for nodes connected by constructed negative hyperedges. Cluster assignment learning refines these embeddings by aligning them with the final cluster structure, obtaining the final clustering assignments. Extensive experimental results demonstrate the consistent superiority of the proposed algorithm across various hypergraph datasets.

Acknowledgments

This work was supported by the National Natural Science Foundation of China [No.62572002, and No.62272001] and Natural Science Foundation of Anhui Province of China [No.2508085MF159].

References

- [1] Sameer Agarwal, Jitendra Lim, Lih Zelnik-Manor, Pietro Perona, David J. Kriegman, and Serge J. Belongie. 2005. Beyond Pairwise Clustering. In *Proceedings of the 2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition*. 838–845.
- [2] Alessia Antelmi, Gennaro Cordasco, Mirko Polato, Vittorio Scarano, Carmine Spagnuolo, and Dingqi Yang. 2023. A Survey on Hypergraph Representation Learning. *Comput. Surveys* 56, 1 (2023), 24:1–24:38.
- [3] Devanshu Arya, Deepak K. Gupta, Stevan Rudinac, and Marcel Worring. 2020. HyperSAGE: Generalizing Inductive Representation Learning on Hypergraphs. arXiv:2010.04558.
- [4] Claude Berge. 1984. *Hypergraphs: combinatorics of finite sets*. Vol. 45. Elsevier.
- [5] Alain Bretto. 2013. *Hypergraph Theory: An Introduction*. Springer.
- [6] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. 2020. A Simple Framework for Contrastive Learning of Visual Representations. In *Proceedings of the 37th International Conference on Machine Learning*. 1597–1607.
- [7] Martina Contisciani, Federico Battiston, and Caterina De Bacco. 2022. Inference of Hyperedges and Overlapping Communities in Hypergraphs. *Nature Communications* 13 (2022), 7229.
- [8] Dheeru Dua and Casey Graff. 2017. UCI Machine Learning Repository. <http://archive.ics.uci.edu/ml>.
- [9] Yifan Feng, Haoran You, Zizhao Zhang, Rongrong Ji, and Yue Gao. 2019. Hypergraph Neural Networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 3558–3565.
- [10] Zijin Feng, Miao Qiao, and Hong Cheng. 2023. Modularity-based Hypergraph Clustering: Random Hypergraph Model, Hyperedge-cluster Relation, and Computation. *Proceedings of the ACM on Management of Data* 1, 3 (2023), 215:1–215:25.
- [11] Zijin Feng, Miao Qiao, Chengzhi Piao, and Hong Cheng. 2025. On Graph Representation for Attributed Hypergraph Clustering. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 59:1–59:26.
- [12] Aditya Grover and Jure Leskovec. 2016. node2vec: Scalable Feature Learning for Networks. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 855–864.
- [13] John A. Hartigan and Manchek A. Wong. 1979. A K-Means Clustering Algorithm. *Journal of the Royal Statistical Society Series C: Applied Statistics* 28, 1 (1979), 100–108.
- [14] Jie Huang, Chuan Chen, Fanghua Ye, Jiajing Wu, Zibin Zheng, and Guohui Ling. 2019. Hyper2vec: Biased Random Walk for Hyper-network Embedding. In *Proceedings of the 24th International Conference on Database Systems for Advanced Applications*. 273–277.
- [15] Igor Kabiljo, Brian Karrer, Mayank Pundir, Sergey Pupyrev, and Alon Shalita. 2017. Social Hash Partitioner: A Scalable Distributed Hypergraph Partitioner. *Proceedings of the VLDB Endowment* 10, 11 (2017), 1418–1429.
- [16] Barakeel Fanseu Kamhoua, Lin Zhang, Kaili Ma, James Cheng, Bo Li, and Bo Han. 2021. HyperGraph Convolution Based Attributed HyperGraph Clustering. In *The 30th ACM International Conference on Information and Knowledge Management*. 453–463.
- [17] George Karypis, Rajat Aggarwal, Vipin Kumar, and Shashi Shekhar. 1999. Multi-level Hypergraph Partitioning: Applications in VLSI Domain. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 7, 1 (1999), 69–79.
- [18] Zheng Tracy Ke, Feng Shi, and Dong Xia. 2020. Community Detection for Hypergraph Networks via Regularized Tensor Power Iteration. arXiv:1909.06503.
- [19] Harold W. Kuhn. 1955. The Hungarian Method for the Assignment Problem. *Naval Research Logistics Quarterly* 2, 1–2 (1955), 83–97.
- [20] Dongjin Lee and Kijung Shin. 2023. I'm Me, We're Us, and I'm Us: Tri-Directional Contrastive Learning on Hypergraphs. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 8541–8549.
- [21] Geon Lee, Soo Yong Lee, and Kijung Shin. 2024. Villain: Self-Supervised Learning on Homogeneous Hypergraphs without Features via Virtual Label Propagation. In *Proceedings of the ACM Web Conference 2024*. 594–605.
- [22] Fan Li, Xiaoyang Wang, Dawei Cheng, Wenjie Zhang, Ying Zhang, and Xuemin Lin. 2024. Hypergraph Self-supervised Learning with Sampling-efficient Signals. In *Proceedings of the International Joint Conference on Artificial Intelligence*. 4398–4406.
- [23] Zhao Li, Xin Wang, Jun Zhao, Feng Feng, Zirui Chen, and Jianxin Li. 2025. HySAE: An Efficient Semantic-Enhanced Representation Learning Model for Knowledge Hypergraph Link Prediction. In *Proceedings of the ACM on Web Conference 2025*. 86–97.
- [24] Yong Mi, Hongmei Chen, Zhong Yuan, Chuan Luo, Shi-Jinn Horng, and Tianrui Li. 2024. Fast Multi-view Subspace Clustering with Balance Anchors Guidance. *Pattern Recognition* 145 (2024), 109895.
- [25] Shilin Qu, Weiqing Wang, Yuan-Fang Li, Xin Zhou, and Fajie Yuan. 2026. Hypergraph Node Representation Learning with One-stage Message Passing. *Neural Networks* 193 (2026), 107973.
- [26] Ryan A. Rossi and Nesreen K. Ahmed. 2015. The Network Data Repository with Interactive Graph Analytics and Visualization. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 4292–4293.
- [27] Peter J. Rousseeuw. 1987. Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis. *J. Comput. Appl. Math.* 20 (1987), 53–65.
- [28] Nicolò Ruggeri, Federico Battiston, and Caterina De Bacco. 2024. Framework to generate hypergraphs with community structure. *Physical Review E* 109, 3 (2024), 034309.
- [29] Bernhard Schölkopf, John Platt, and Thomas Hofmann. 2007. Learning with Hypergraphs: Clustering, Classification, and Embedding. In *Advances in Neural Information Processing Systems*. 1601–1608.
- [30] Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Galligher, and Tina Eliassi-Rad. 2008. Collective Classification in Network Data. *AI Magazine* 29, 3 (2008), 93–106.
- [31] Shantanu Thakoor, Corentin Tallec, Mohammad Gheshlaghi Azar, Mehdi Azabou, Eva L. Dyer, Remi Munos, Petar Veličković, and Michal Valko. 2022. Large-Scale Representation Learning on Graphs via Bootstrapping. In *International Conference on Learning Representations (ICLR)*. New York, NY, USA.
- [32] Laurens van der Maaten and Geoffrey Hinton. 2008. Visualizing Data Using t-SNE. *Journal of Machine Learning Research* 9, 86 (2008), 2579–2605.
- [33] Petar Veličković, William Fedus, William L. Hamilton, Pietro Liò, Yoshua Bengio, and R Devon Hjelm. 2019. Deep Graph Infomax. In *International Conference on Learning Representations (ICLR)*. 1–10.
- [34] Qianqian Wang, Bowen Zhao, Zhengming Ding, Xiangdong Zhang, and Quanxue Gao. 2025. A Simple yet Effective Hypergraph Clustering Network. In *Proceedings of the International Joint Conference on Artificial Intelligence*. 6352–6360.
- [35] Siwei Wang, Xinwang Liu, Li Liu, Sihang Zhou, and En Zhu. 2023. Late Fusion Multiple Kernel Clustering With Proxy Graph Refinement. *IEEE Transactions on Neural Networks and Learning Systems* 34, 8 (2023), 4359–4370.
- [36] Tianxin Wei, Yuning You, Tianlong Chen, Yang Shen, Jingrui He, and Zhangyang Wang. 2022. Augmentations in Hypergraph Contrastive Learning: Fabricated and Generative. In *Advances in Neural Information Processing Systems* 35. 1677–1691.
- [37] Yuze Wu, Shiyang Lan, Zhiling Cai, Mingjian Fu, Jinbo Li, and Shiping Wang. 2025. SCHG: Spectral Clustering-guided Hypergraph Neural Networks for Multi-view Semi-supervised Learning. *Expert Systems with Applications* 277 (2025), 127242.
- [38] Lianghao Xia, Chao Huang, Yong Xu, Jiashu Zhao, Dawei Yin, and Jimmy Xiangji Huang. 2022. Hypergraph Contrastive Collaborative Filtering. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Madrid, Spain). New York, NY, USA, 1755–1765.
- [39] Xin Xia, Hongzhi Yin, Junliang Yu, Qinyong Wang, Lizhen Cui, and Xiangliang Zhang. 2021. Self-Supervised Hypergraph Convolutional Networks for Session-based Recommendation. In *Proceedings of the Thirty-Fifth AAAI Conference on Artificial Intelligence*. 4503–4511.
- [40] Jing Xiao, Zhi-Wei Ma, Jian Cao, and Xiao-Ke Xu. 2025. Hypergraph Community Detection With Fuzzy Memberships. *IEEE Transactions on Fuzzy Systems* 33, 9 (2025), 3253–3266.
- [41] Lihong Xiao, Xia Wu, Lie-Wei Zheng, Li Wang, Lei Wang, Daijiang Zhu, Ding-gang Shen, and Tianming Liu. 2020. Multi-Hypergraph Learning-Based Brain Functional Connectivity Analysis in fMRI Data. *IEEE Transactions on Medical Imaging* 39, 5 (2020), 1746–1758.
- [42] Junyuan Xie, Ross Girshick, and Ali Farhadi. 2016. Unsupervised Deep Embedding for Clustering Analysis. In *Proceedings of the 33rd International Conference on Machine Learning*. 478–487.
- [43] Naganand Yadati, Madhav Nimishakavi, Prateek Yadav, Vikram Nitin, Anand Louis, and Partha Talukdar. 2019. HyperGCN: A New Method for Training Graph Convolutional Networks on Hypergraphs. In *Advances in Neural Information Processing Systems* 32. 1511–1522.
- [44] Yuguang Yan, Yuanlin Chen, Shibo Wang, Hanrui Wu, and Ruichu Cai. 2024. Hypergraph Joint Representation Learning for Hypervertices and Hyperedges via Cross Expansion. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 9232–9240.
- [45] Yiyang Yang, Sucheng Deng, Juan Lu, Yuhong Li, Zhiguo Gong, Leong Hou U, and Zhifeng Hao. 2021. GraphLSHC: Towards Large Scale Spectral Hypergraph Clustering. *Information Sciences* 544 (2021), 117–134.
- [46] Yuning You, Tianlong Chen, Yongduo Sui, Ting Chen, Zhangyang Wang, and Yang Shen. 2020. Graph Contrastive Learning with Augmentations. In *Advances in Neural Information Processing Systems* 33. New York, NY, USA, 5812–5823.
- [47] Jun Yu, Dacheng Tao, and Meng Wang. 2012. Adaptive Hypergraph Learning and its Application in Image Classification. *IEEE Transactions on Image Processing* 21, 7 (2012), 3262–3272.
- [48] Erchuan Zhang, David Suter, Giang Truong, and Syed Zulqarnain Gilani. 2022. Sparse Hypergraph Community Detection Thresholds in Stochastic Block Model. In *Advances in Neural Information Processing Systems* 35. 25039–25052.
- [49] Wenqian Zhang, Zhengyi Yang, Dong Wen, Wentao Li, Wenjie Zhang, and Xuemin Lin. 2025. Accelerating Core Decomposition in Billion-Scale Hypergraphs. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 6:1–6:27.
- [50] Tianxiang Zhao, Youqing Wang, Jinlu Wang, Jiapu Wang, Mingliang Cui, Junbin Gao, and Jipeng Guo. 2025. Hybrid-Collaborative Augmentation and Contrastive Sample Adaptive-Differential Awareness for Robust Attributed Graph Clustering. arXiv:2510.02731

- [51] Tianxiang Zhao, Youqing Wang, Shilong Xu, Tianchuan Yang, Junbin Gao, and Jipeng Guo. 2026. Dual-level noise augmentation for graph clustering with triplet-wise contrastive learning. *Pattern Recognition* 172 (2026), 112463.
- [52] Sihang Zhou, Xinwang Liu, Miaomiao Li, and et al. . 2020. Multiple Kernel Clustering With Neighbor-Kernel Subspace Segmentation. *IEEE Transactions on Neural Networks and Learning Systems* 31, 4 (2020), 1351–1362.

A Details of datasets

To evaluate the performance of the algorithms, we conduct experiments on eight datasets. These datasets include co-citation networks (Cora, Citeseer, and Pubmed) [30]; co-authorship networks (Cora-Coauthor and DBLP-Coauthor) [26]; a computer vision dataset (NTU2012) [6]; and two datasets from the UCI Machine Learning Repository (20NewsW100 and Mushroom) [8]. Statistics information about these datasets is shown in Table 1.

For the co-citation network, nodes represent academic papers. A hyperedge represents a set of papers that are all cited together by a common publication. For the co-authorship network, nodes also represent papers. A hyperedge represents all papers that are written by the same author [20]. For co-citation and co-authorship datasets, node features are bag-of-words representations derived from the paper abstracts, and node labels correspond to their research fields. The preprocessed hypergraphs for Cora, Citeseer, Pubmed, Cora-Coauthor, and DBLP-Coauthor datasets are publicly available with the official implementation of HyperGCN [20, 43]. For the visual dataset NTU2012, the hypergraph construction follows the setting described in [9], with node features extracted by a pre-trained convolutional neural network. For 20NewsW100 and Mushroom, node features are TF-IDF representations and categorical descriptions, respectively. In experiments, we delete isolated nodes in datasets. Since our work focuses on unsupervised node clustering, the entire graph structure and all node features are utilized during training, and the ground-truth labels are used exclusively for post-hoc evaluation.

B Implementation details

All experiments are performed on a system equipped with an NVIDIA GeForce RTX 4090 GPU (48GB memory), 64GB of RAM, and an Intel Core i9-13900K CPU. CAHC is implemented using PyTorch 2.4.1 and PyTorch Geometric 2.6.1.

B.1 Hyperparameters

CAHC is trained using the Adam optimizer and has several parameters. Several hyperparameters were fixed across all experiments, introduced as follows. The number of encoder layers was set to 1. Within the enhanced encoder, the number of attention heads was fixed at 4, and the dimension of each attention head was set to 128. In the appendix D.1, we have added an independent sensitivity analysis regarding the number and dimension of attention heads. The contrastive temperature τ was set to 0.5.

Some parameters take different values on different datasets, such as the embedding dimension (D), learning rate (LR), cluster assignment learning rate (Ass.LR), weight decay, augmentation rates (p_f , p_m), and the number of epochs for representation learning (Pre.E) and cluster assignment learning (Ass.E), respectively. The values of these parameters for each dataset are provided in Table 4.

B.2 Baseline details

To evaluate the performance of CAHC, we compare it with seven baselines, introduced as follows:

1) Node2vec [12]. Node2vec is a graph representation learning framework that generates node sequences through a biased random walk and learns node embeddings in conjunction with the skip-gram model.

2) DGI [33]. DGI is an unsupervised GCN framework that learns node embeddings by maximizing the mutual information between local node representations and the global graph summary.

3) RAGC [50]. RAGC is a robust attributed graph clustering algorithm. It primarily comprises a hybrid collaborative augmentation module and a contrastive sample adaptive-differential awareness module.

4) Hyper2vec [14]. Hyper2vec is a representation learning algorithm that extends the random walk paradigm to hypergraphs. It designs a biased second-order random walk strategy to sample node sequences to capture the high-order structural information, and combines it with the skip-gram framework to learn the final node embeddings.

5) TriCL [20]. TriCL is a three-level contrastive framework that captures multi-granular structural information from nodes, hyperedges, and their membership interactions. k-means is employed on the node embeddings to generate clusters.

6) SE-HSSL [22]. SE-HSSL incorporates sampling-free CCA-based objectives at both node and group levels, alongside a novel hierarchical membership contrastive objective. k-mean is employed on the node embeddings to generate clusters.

Referring to [20, 21], for self-supervised methods designed for graphs (DGI and RAGC), we first convert hypergraphs to graphs via clique expansion and apply these methods to the converted graphs. For embedding-based baseline methods, the learned embeddings are fed into the k-means algorithm [13] for clustering. For all baselines, we used the publicly available code from the original paper. Since both CAHC and the baselines are unsupervised, no labeled validation data was available for model selection. Thus, for each baseline, we either adopted their default hyperparameter settings or conducted a grid search to find configurations that work well on datasets, following the guidelines from papers. The parameter settings for the baselines designed for graphs and hypergraphs are described as follows, respectively.

For the baselines designed for graphs, Node2vec, DGI, and RAGC—we first transformed the hypergraphs into standard graphs using clique expansion, where each hyperedge is converted into a clique. Their specific settings are as follows. For Node2vec, we used the default hyperparameters; specifically, we set the number of walks per node to 10, the length of each walk to 80, the window size to 5, and the learning rate to 0.05, with parameters p and q both set to 1. For DGI, we used the PReLU activation function and set the learning rate to 0.001, as given by default. For RAGC, the hidden dimension was set to 1500, and the numbers of local and global diffusion steps (t_l , t_m) were both set to 3. The confidence threshold τ was 0.9, the positive sample weight β was 0.9, and the negative sample weight γ was 1.0.

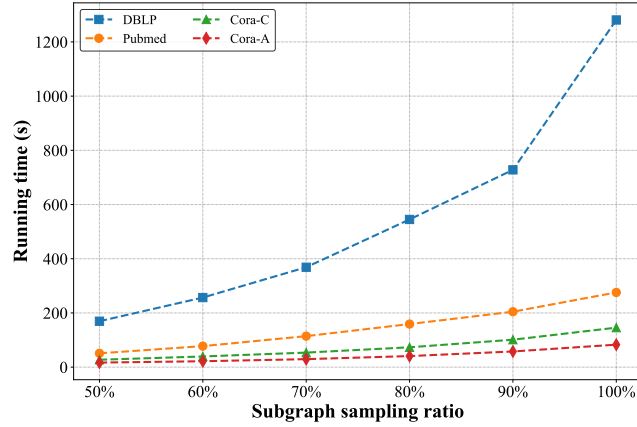
For the baselines designed for hypergraphs, we used the following settings. For Hyper2vec, the number of walks was set to 10

Table 4: Key hyperparameter settings for our model (CAHC). Fixed parameters are detailed in the text.

Dataset	p_f	p_m	D	LR	Ass. LR	WD	Pre. E	Ass. E
Cora-C	0.4	0.4	512	5e-4	5e-4	5e-4	180	120
Citeseer	0.4	0.4	2048	1e-4	1e-5	5e-5	200	80
Pubmed	0.1	0.4	1024	5e-5	5e-5	5e-5	150	60
Cora-A	0.3	0.2	768	1e-4	1e-5	1e-5	100	140
DBLP	0.4	0.2	256	8e-4	8e-4	5e-5	260	100
20NewsW100	0.1	0.2	512	1e-3	1e-4	5e-4	80	30
Mushroom	0.4	0.2	128	1e-4	1e-4	5e-4	120	80
NTU2012	0.4	0.2	512	1e-5	1e-5	5e-5	60	40

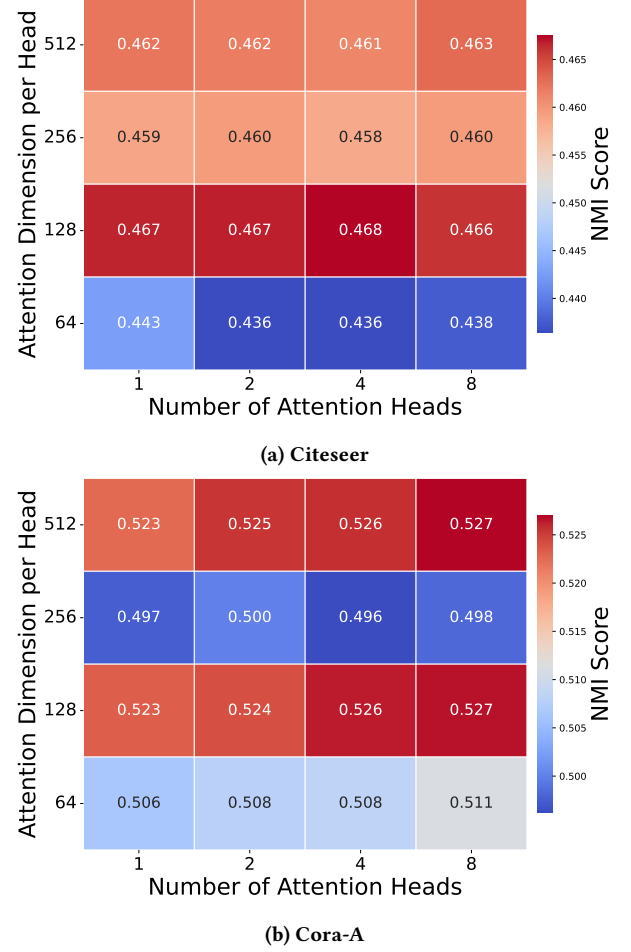
with a length of 20, the window size was 5, and the parameters p and q were both set to 1. For both TriCL and SE-HSSL, we strictly adhered to the default hyperparameter settings provided in their respective official implementations and original papers, without any modifications.

C Scalability

**Figure 6: Scalability of CAHC on the DBLP, cora-C, Pubmed, and Cora-A datasets.**

To evaluate the scalability of CAHC, we randomly sample hypergraphs of different sizes from the DBLP, Cora-C, Pubmed, and Cora-A datasets and test CAHC on these extracted hypergraphs. For each dataset and each sampling ratio in 50%, 60%, 70%, 80%, 90%, 100%, we extract hypergraphs containing the corresponding ratio of nodes from the dataset, with hyperedges that include the extracted nodes. The runtime of CAHC on these datasets is shown in Figure 6.

Figure 6 shows that, as the percentage of nodes increases, the runtime of CAHC improves on datasets. This is because the computational complexity is directly related to the number of nodes, hyperedges, and degrees in the hypergraph. As the hypergraph grows in size, the computational cost of each message passing increases. For instance, as the number of nodes and hyperedges in the network increases, the average node degree rises, causing the

**Figure 7: Parameter analysis of the number of heads and attention dimension on the Citeseer and Cora-A datasets. The color intensity corresponds to the NMI score.**

hypergraph neural network to aggregate information from more neighbors, thus increasing computational cost.

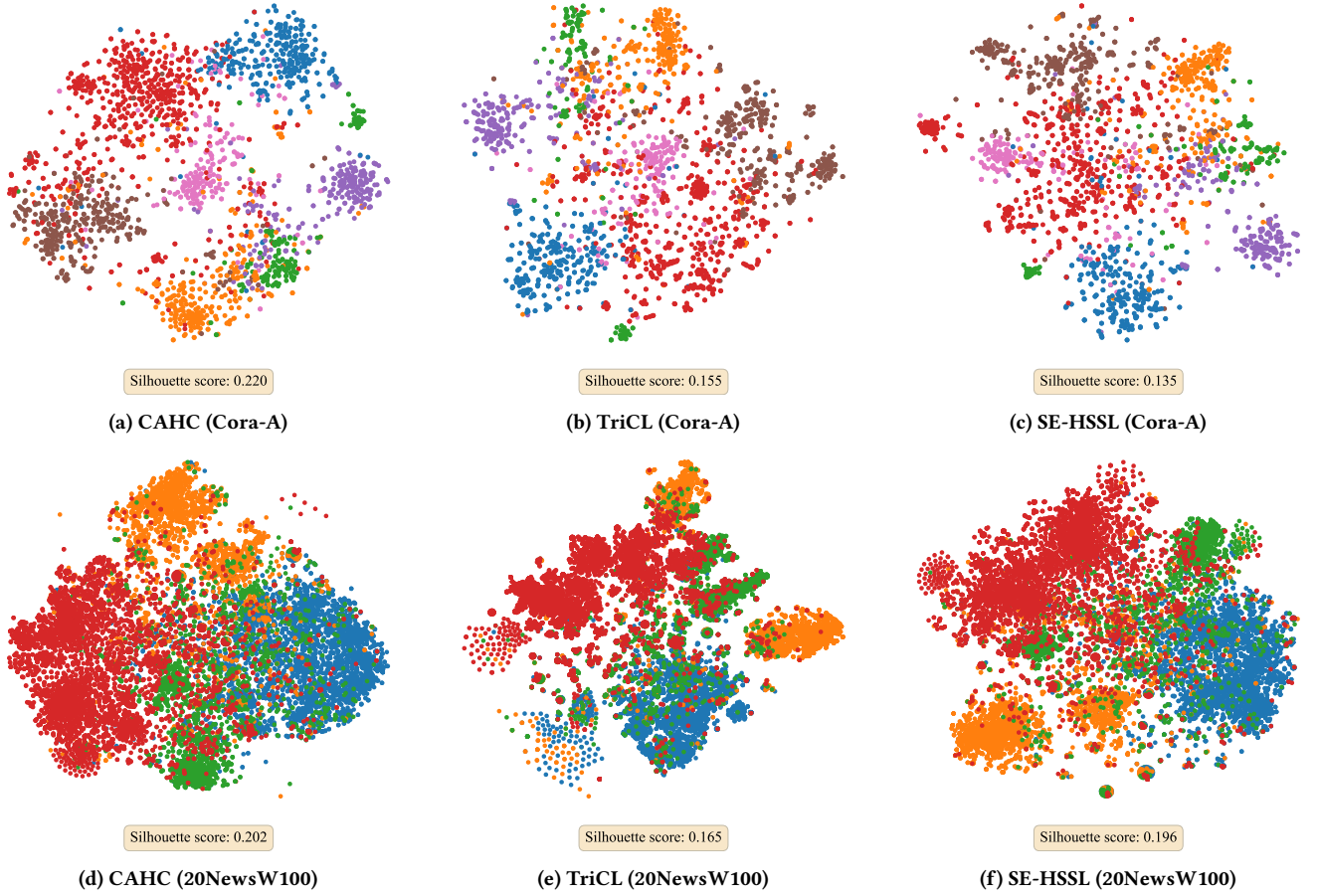


Figure 8: t-SNE visualization of learned node embeddings on the Cora-A (top row) and 20newW100 (bottom row) datasets. The visualization compares the clustering quality of our proposed model against baselines. Points represent nodes, colored by their ground-truth labels.

D Additional experiments

D.1 Backbone encoder

To evaluate the number of attention heads and the dimension of each head on the performance of CAHC, we conduct a parameter analysis on the Citeseer and Cora-Ar datasets. The number of heads was varied within the set $\{1, 2, 4, 8\}$, while the attention dimension was selected from $\{64, 128, 256, 512\}$. The performance, measured by Normalized Mutual Information (NMI), is visualized via heatmaps for the Citeseer and Cora-A datasets in Figure 7.

Figure 7 shows that CAHC on both datasets achieves the best performance when the dimensionality is moderate (128), and the model performance seems to be more sensitive to the choice of attention dimension than the choice of the number of heads. Furthermore, we observe that a multi-head configuration generally outperforms a single-head setting. This validates the effectiveness of the multi-head mechanism, suggesting that allowing the model to attend to information from different representation subspaces simultaneously may be beneficial for capturing diverse features and ultimately enhancing clustering performance.

D.2 Qualitative analysis

To intuitively compare the quality of the learned embeddings of algorithms, we employed t-SNE [32] for visualizing the node representations learned by CAHC, TriCL, and SE-HSSL. Figure 8 presents the t-SNE visualizations of node embeddings on the Cora-A and 20NewsW100 datasets.

Figure 8 shows that while TriCL and SE-HSSL yield disorganized results with no discernible clusters, CAHC demonstrates compact and well-separated clusters. This visual assessment is confirmed quantitatively by the Silhouette score [27]. One potential reason TriCL and SE-HSSL fail to produce separable embeddings is that they lack clustering guidance. On the Cora-A dataset, CAHC achieves the highest Silhouette score, indicating the best cluster quality. These conclusions are consistent with results in Table 2.